

1. (b) Data and a pointer to the next node.
2. (c) NULL.
3. (b) Self-referential structure
4. (b) head == NULL
5. (b) newnode $\rightarrow$ next = head; head = new Node;
6. (b) temp = head; head = head $\rightarrow$ next; free(temp);
7. (c) Traversing from tail to head.
8. (b) n.
9. (c) Copying the data of $P \rightarrow$ next into P then deleting $P \rightarrow$ next.
10. (b) while (P != NULL)
11. (b) P $\rightarrow$ next == NULL
12. (b) A Stack of visited nodes
13. (c) Three
14. (b) Exactly one node
15. (b) P = (struct node*) malloc(sizeof(struct node));
16. (b) Avoid special-case handlings for insertion and deletion at the head.
17. (c) The first node.
18. (b) It does not support random (indexed) access.
19. (b) Traversing access to all nodes, causing a memory leak.
20. (b) A Slow Pointer and a fast Pointer.
21. (c) The last node.
22. (b) Nodes may be scattered anywhere in memory.
23. Ans:-

```
struct node {
    int data;
    struct Node* next;
};
```

24. Ans:- The code prints the sum of each node and its next node;
 $\rightarrow 10 + 20 = 30$
 $\rightarrow 20 + 30 = 50$
 $\rightarrow 30 + 40 = 70$
 Output is $\rightarrow 30\ 50\ 70$.

25. Ans:- Error:- head is passing by value, So changes to the head pointer won't affect the original list. Also, an empty list case needs handling correctly:-

```
void insertEnd (Struct node** head, int x)
```

```
{
    Struct node* t = (Struct node*) malloc
    sizeof (Struct node);
```

```
    t->data = x;
```

```
    t->next = NULL;
```

```
    if (*head == NULL) {
```

```
        *head = t;
```

```
    }
    return;
```

```
}
```

```
Struct node* p = *head;
```

```
while (p->next != NULL) {
```

```
    p = p->next;
```

```
    p->next = t;
```

```
}
```

26. Struct node* Reverse (Struct node* head) {

```
    Struct node* prev = NULL;
```

```
    Struct node* curr = head;
```

```
    Struct node* next = NULL;
```

```
    while (curr != NULL) {
```

```
        next = curr->next;
```

```
        curr->next = prev;
```

```
        prev = curr;
```

```
        curr = next;
```

```
}
```

```
    return prev;
```

```
}
```

27. Suppose, the list is 10 → 20 → 30 → 40 → NULL

1) I want to delete 30.

2) Then, first 20 node's next pointer needs to be pointed to 40.

Before $\rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow \text{NULL}$
 Prev del next

After $\rightarrow 10 \rightarrow 20 \rightarrow \text{NULL}$
 ↑

20 deleted

Prev $\rightarrow$ next = del $\rightarrow$ next;
 free (del);

28. Initial List: -

$5 \rightarrow 8 \rightarrow 12 \rightarrow 3 \rightarrow \text{NULL}$

(i) $7 \rightarrow 5 \rightarrow 8 \rightarrow 12 \rightarrow 3 \rightarrow \text{NULL}$

(ii) $7 \rightarrow 5 \rightarrow 8 \rightarrow 3 \rightarrow \text{NULL}$

(iii) $7 \rightarrow 5 \rightarrow 8 \rightarrow 3 \rightarrow 9 \rightarrow \text{NULL}$

Final answer: $7 \rightarrow 5 \rightarrow 8 \rightarrow 3 \rightarrow 9 \rightarrow \text{NULL}$